FORKING: OVER-ENCODING CAUSES OVER-MEMORIZATION UNDER DATA REPLAY

Anonymous authors
Paper under double-blind review

ABSTRACT

This paper studies a generalization failure discovered during an automated architecture search for NanoGPT, which we call *forking*. When the training data is repeatedly consumed, models with an over-encoding n-gram memory branch exhibit a clear separation between training and validation loss, and this separation often grows in a staircase pattern aligned with replay epoch boundaries. We study this phenomenon in a controlled vanilla NanoGPT setting. Our experiments show that over-encoding partitions the optimization process into finer context-specific subspaces. Repeated updates then sharpen the observed continuations within these subspaces while damaging alternative continuations that remain plausible but receive insufficient coverage. We further observe the same forking phenomenon in a DeepSeek-style model with an Engram-like memory module. In a continued-pretraining stress test on small ID and OOD corpora, the gap reopens when the n-gram table remains trainable; freezing the table substantially reduces but does not eliminate the effect.

1 INTRODUCTION

Language-model agents are increasingly used to propose, execute, and evaluate research ideas. Systems for automated scientific discovery and model improvement extend this process from individual coding tasks to iterative experimental search (Lu et al., 2024; Recursive, 2026). Such searches can improve a target metric, but they can also reveal behavior that was not part of the original objective. This paper studies one such discovery in a NanoGPT autoresearch workflow.

The agent's task was to modify the model architecture and trainer to minimize next-token validation bits per byte (BPB). During a small-data experiment, we observed an unexpected pattern: Training and validation losses initially improved together, But once the loader began replaying previously seen text, the curves separated rapidly near the epoch boundary. Training loss continued to fall, while validation loss stopped improving or even increased. The gap then grew in steps over subsequent passes. We call this replay-aligned divergence *forking* (Figure 1).

This behavior resembles overfitting: training loss continues to decrease while validation loss deteriorates, a pattern previously observed in language models trained on repeated data (Hernandez et al., 2022). However, such a sharp divergence is not an inevitable consequence of data reuse; previous studies show that language models can benefit from several passes over a finite corpus (Muennighoff et al., 2023). Through a series of controlled experiments, we trace forking to the interaction between data replay and *over-encoding*, which enriches token representations with additional embeddings of local token combinations (Huang et al., 2025). In our model, an n-gram module uses the most recent n tokens, including the current token, to retrieve a learned embedding from a hash-indexed table. This embedding is injected into the Transformer backbone to help predict the next token. Across the configurations we test, adding this module produces a pronounced training–validation gap once the training data are replayed.

To understand why over-encoding promotes overfitting under replay, we examine how the gap depends on context frequency. Low-frequency n-gram contexts account for a substantial share of the additional gap, and the average gap follows an approximate power-law relationship with context frequency over the measured range. We propose that the n-gram table makes optimization more local by assigning distinct learned representations to different contexts. These representations allow the model to fit local prediction problems more independently, making it easier to memorize

the continuations observed in training. For rare contexts, only a few valid continuations may be represented in the corpus. Replaying the same examples reinforces these continuations without expanding their coverage, potentially suppressing alternatives needed at validation time. We develop this explanation through frequency-dependent interventions, a model of continuation coverage, and an analysis of replay dynamics.

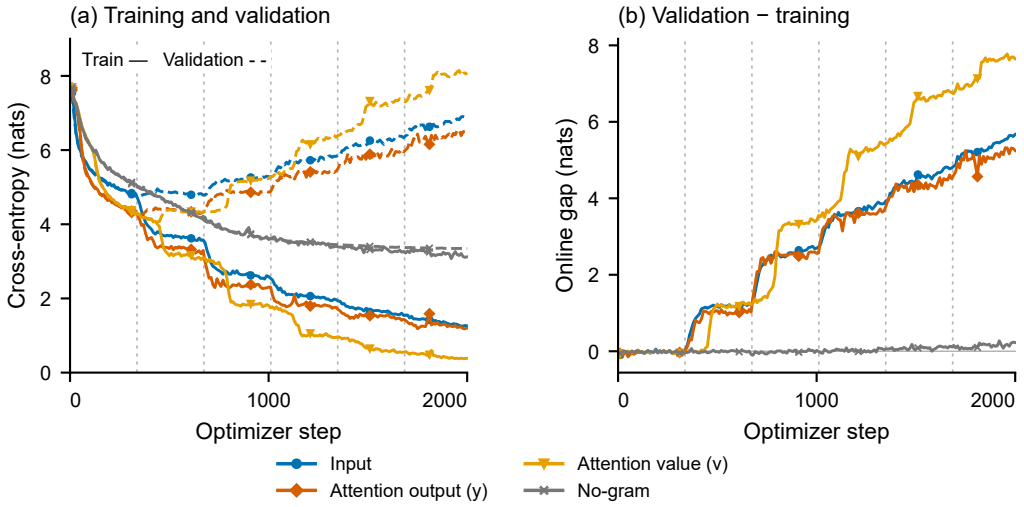


Figure 1: Forking under fixed data replay. (a) Online training loss (solid) and fixed validation loss (dashed). (b) Online gap, defined as fixed validation loss minus current-batch training loss at the same logged step. Colors and markers distinguish 3 over-encoding and 1 no-over-encoding configurations .The details of four configurations are illustrated in Figure 2. All runs use seed 42 and 2000 updates; enabled n-gram tables use a learning rate 128 times that of the backbone. Vertical dotted lines mark 337-step replay boundaries. Curves show raw logged values without smoothing.

2 Related Work

NanoGPT, autoresearch, and recursive model improvement. NanoGPT provides a compact implementation for training GPT-style language models (Karpathy, 2022). The autoresearch workflow places a language-model agent in a repeated cycle of modifying a trainer, running a budgeted experiment, and evaluating validation BPB (Karpathy, 2026). Related automated-research reports explore how agents can use earlier experiments to propose and combine improvements to language-model training (Recursive, 2026). Our work begins within this kind of search, but focuses on an unexpected training phenomenon and the conditions under which an automatically selected modification fails to generalize.

N-gram augmentation, over-encoding, and conditional memory. N-Grammer augments Transformers with latent n-gram representations, using discrete latent features to construct additional context-dependent representations (Roy et al., 2022). The Over-Tokenized Transformer separates the capacity of input encoding from the output vocabulary and augments input representations with projected multi-order n-gram embeddings (Huang et al., 2025). Its experiments show that increasing sparse input vocabulary can improve language-model loss over the tested range. Engram develops scalable lookup-based conditional memory as a complementary form of sparsity, combining n-gram retrieval with learned integration into the network (Cheng et al., 2026). These methods differ in their keys, lookup mechanisms, and injection routes, but all expand the model's access to local context information. We examine a potential cost of this capacity when finite training data are replayed.

Training with repeated data. Data-constrained scaling studies show that repeated tokens can remain useful, while their marginal value decreases as repetition increases (Muennighoff et al., 2023). Hernandez et al. (2022) study how repeated subsets affect language-model learning, including

increases in test loss during training and changes in copying-related behavior. These results establish that replay can alter both generalization and internal computation. Forking adds a component-specific case: rapid, replay-aligned divergence associated with local n-gram memory, together with interventions that test the role of context frequency and memory access.

Good–Turing estimation and continuation coverage. Good–Turing estimation uses frequency-of-frequencies statistics to estimate the probability mass of outcomes absent from a finite sample (Good, 1953; Gale & Sampson, 1995). Its missing-mass estimate is the fraction of observations belonging to outcomes seen exactly once. Applied conditionally to an observed context, this estimates the probability assigned to continuations not represented in its training occurrences. This provides a statistical basis for studying incomplete continuation coverage. Relating that missing mass to a model's loss gap requires additional assumptions about how training changes predictions for seen and unseen continuations.

3 Forking and Its Empirical Structure

3.1 Minimal setting

We use a NanoGPT backbone with 8 layers, 6 attention heads, a hidden dimension of 768, a vocabulary of 8192 tokens, and a sequence length of 2048. We set separate lookup table for bigram and trigram, and we compare three injection routes: input, attention value (v), and attention output (y) (illustrated in Figure 2), together with a no-gram baseline. The input route is the default unless otherwise stated. Detailed definitions of the three routes are deferred to Appendix B.

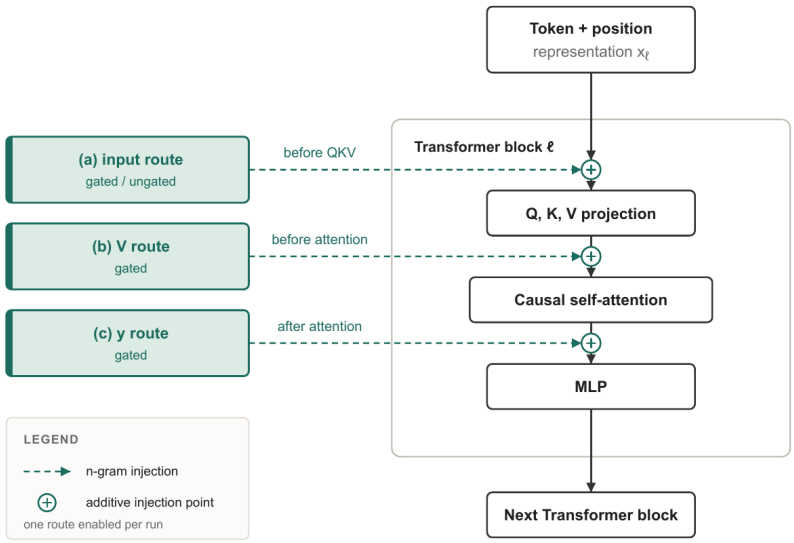


Figure 2: Three n-gram injection routes. Retrieved embeddings are added to token representations before attention (input), to the value vectors used by attention (v), or to the attention output before the MLP (y). Each experimental configuration uses one route; the no-gram baseline removes the n-gram branches.

The backbone uses AdamW with coefficients $(0.8, 0.95)$ and a learning rate of 6×10^{-4}. Table parameters use RMSProp with first-moment coefficient zero, second-moment decay 0.99, and a learning rate of 0.0768, or 128 times the backbone learning rate. Learning rates are warmed up for 100 steps and then held constant. One epoch over the fixed single-shard training set takes approximately 337 steps. Unless otherwise stated, runs use seed 42. Appendix B provides numerical precision and implementation details.

Training loss (L_{tr}) is the mean next-token cross-entropy on the current training batch, computed before the parameter update. Validation loss (L_{va}) is the mean next-token cross-entropy on fixed held-out validation batches, evaluated after the update. At each evaluation step, we define the online gap as $G = L_{\mathrm{va}} - L_{\mathrm{tr}}$.

3.2 Unique features of the phenomenon

Figure 1 compares our four experiments. All three n-gram configurations show pronounced forking in this setting.

Forking is characterized by the following pattern: during the first epoch, the training and validation loss curves remain relatively close; once the same text is replayed, training loss continues to decrease without a corresponding improvement in validation loss, and the validation–training gap keeps widening. A key feature that distinguishes this phenomenon from ordinary overfitting is that the gap widens mainly at the beginning of each replay epoch, primarily through a rapid decrease in training loss, indicating strong overfitting. The degree of overfitting increases in a staircase pattern over successive epochs, which links the loss dynamics to the replay schedule. The phenomenon is not specific to this minimal architecture: a 12-layer DeepSeek-style model with multi-layer Engram input over-encoding reproduces forking and responds to low-frequency read masking in the same way (Appendix G).

3.3 Empirical dynamics laws

All experiments below use input injection, seed 42, and a table learning rate 128 times the backbone learning rate when a table is enabled. For loss, the frequency analysis uses fixed probe loss for both training and validation, while the remaining experiments use online training loss and a fixed validation probe.

Context frequency. A finite corpus does not fully represent the continuation distribution of each context. For example, a context whose population distribution assigns equal probability to C and D may appear three times with C and once with D in training. Fitting these observations can lead a model to underpredict D on independent data.

We enable both bigram and trigram tables, each with 2^{20} rows, and measure context-matched fixed-probe gaps at step 1000. Let $F(i)$ denote the count of context i in the unrepeated training corpus. The same run is analyzed using bigram and trigram context groupings. We display 12 geometric frequency bins and fit seven token-mass-pooled bins with equal weight across fitted bins. Over the fitted range, Figure 3 shows

$$G_{\mathrm{bigram}}(F) \propto F^{-0.253}, \qquad G_{\mathrm{trigram}}(F) \propto F^{-0.318}. \tag{1}$$

Low-frequency contexts have larger average gaps, and the explanation of the power law relationship refers to section 4.1.

Table capacity. We vary the number of table rows R in separate bigram-only and trigram-only models, holding the single-shard training corpus fixed and measuring the online gap at step 1000. Above a lower limit, the gap generally increases with table capacity, and growth slows toward the upper end of the measured range, particularly for bigrams (Figure 4). We describe this trend using the empirical fit

$$G(R) = 0.02 + AR^{\beta}, \tag{2}$$

where 0.02 is the approximate no-gram reference used in the original analysis. We fit $\log[G(R) - 0.02]$ against $\log R$, including all tested 25 bigram points and 24 trigram points. The fitted exponents are $\beta = 0.437$ and 0.745, with log-space $R^2 = 0.9715$ and 0.9893, respectively.

Training corpus size. We use a trigram-only model with 2^{20} table rows and vary the training corpus size D across 20 conditions from approximately $0.125D_1$ to $20D_1$. Here, D_1 is the single-shard training budget of the minimal setting: 337 batches, or 49,692,672 prediction tokens. Each run completes three epochs, with the same four-batch validation probe.

Increasing the corpus can introduce new contexts and continuations and increase the counts of existing contexts; at a fixed epoch count, it also increases the number of optimizer updates. As a result, the

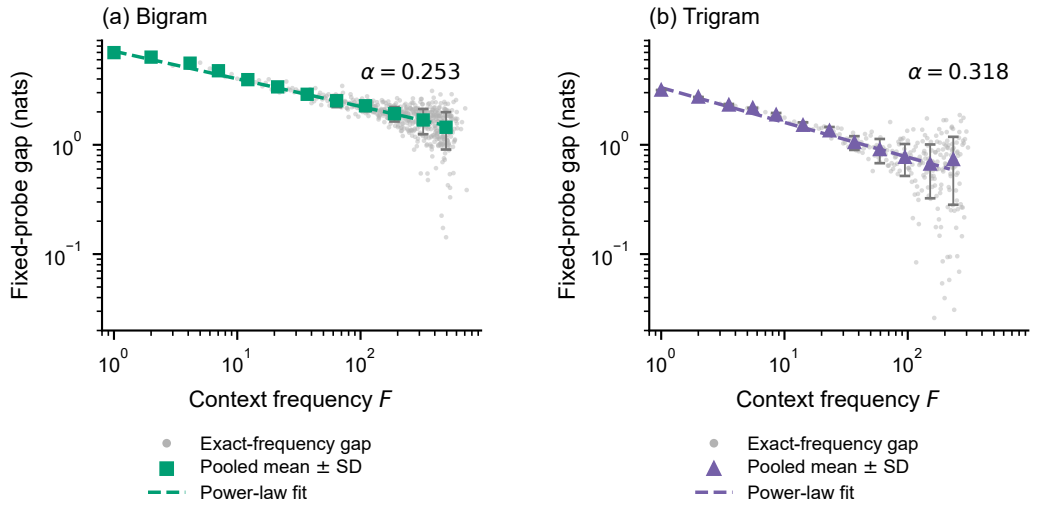


Figure 3: Gap versus context frequency for (a) bigrams and (b) trigrams. Gray points show exact-frequency observations; colored markers show pooled means, with bars indicating within-bin standard deviations. Dashed lines show power-law fits.

gaps at the ends of epochs 2 and 3 generally decrease as the training corpus grows (Figure 5). With validation held fixed, the largest training corpus yields relatively small gaps of 0.13 and 0.22 nats, respectively, at the rightmost points which have 20x training corpus.

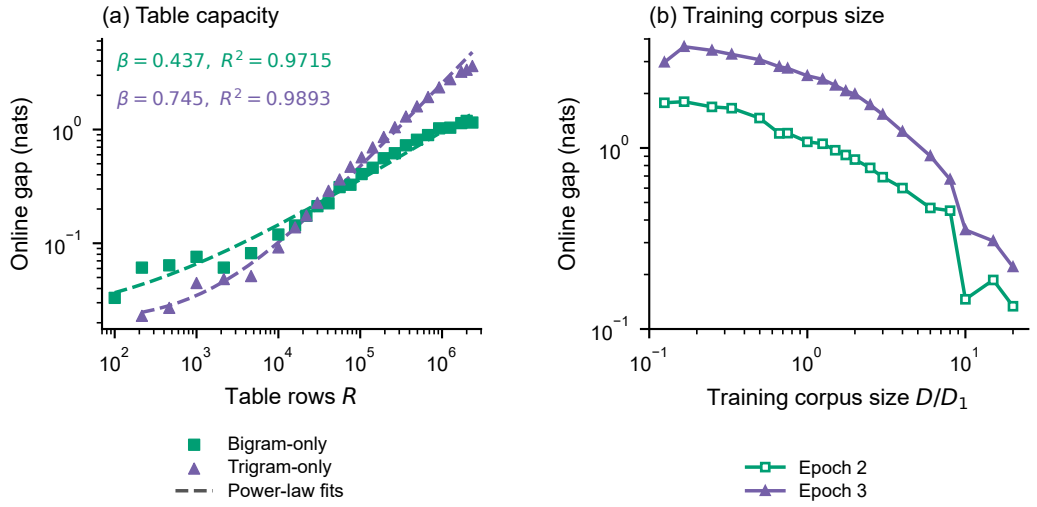


Figure 4: (Left) Gap versus table capacity. Markers show observations; dashed lines show fits within the specified capacity intervals.

Figure 5: (Right) Gap versus normalized training corpus size. Green squares and purple triangles show epoch-2 and epoch-3 endpoints; line segments connect adjacent observations.

Epoch count. We train trigram-only and both-table models for 20 epochs, together with a no-gram control, using the fixed single-shard training corpus with approximately 337 updates per epoch. Each enabled table has 2^{20} rows. The right panel of Figure 6 shows epoch-end gaps from the 20-epoch runs. The gap increases as replay count grows. Separate 2000-step runs in the left panel compare the

both-table model with the no-gram control using dense gap measurements, which show gap growth within replayed epochs with varying increments.

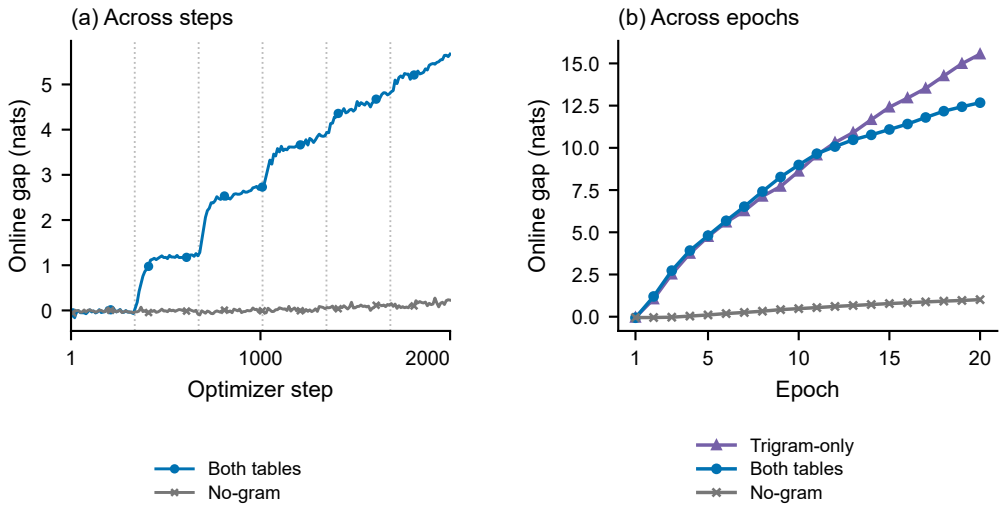


Figure 6: (Left) Online gap versus optimizer step; vertical dotted lines mark replay boundaries. This panel shows the growth pattern of gap within an epoch. (Right) Epoch-end gap versus epoch number.

3.4 Intervention experiments

We study two interventions in the both-table model with input injection. Both use the fixed single-shard training corpus and run for 1000 updates, with interventions starting at step 338, after the first epoch. We measure the online gap using the convention in Section 3.1. The two panels use independent runs, each with its own unmodified control.

Low-frequency retrieval masking We count context frequencies in the unrepeated training corpus and mask the n-gram contribution whenever $F \leq \tau$, applying the rule separately to the bigram and trigram branches with thresholds $\tau \in \{1, 2, 4, 8, 200\}$. The retrieved embedding is set to zero during both training and evaluation, while the corresponding token remains in the training loss. Compared with unmasked training, suppressing low-frequency memory access reduces the gap (Figure 7, left). This experiment supports the the relationship of frequency and gap in 3.3 and provides a simple way to mitigate forking in the tested setting.

Freezing the tables We train both n-gram tables normally during the first epoch, then freeze their parameters while continuing to update the backbone. The learned table contents and context-to-row mapping remain fixed throughout subsequent replay. A pronounced training–validation gap still develops after the tables are frozen (Figure 8, right). Thus, once the tables have learned context representations, other parts of the model can fit to the table which will produce forking, and freezing the tables alone does not prevent it.

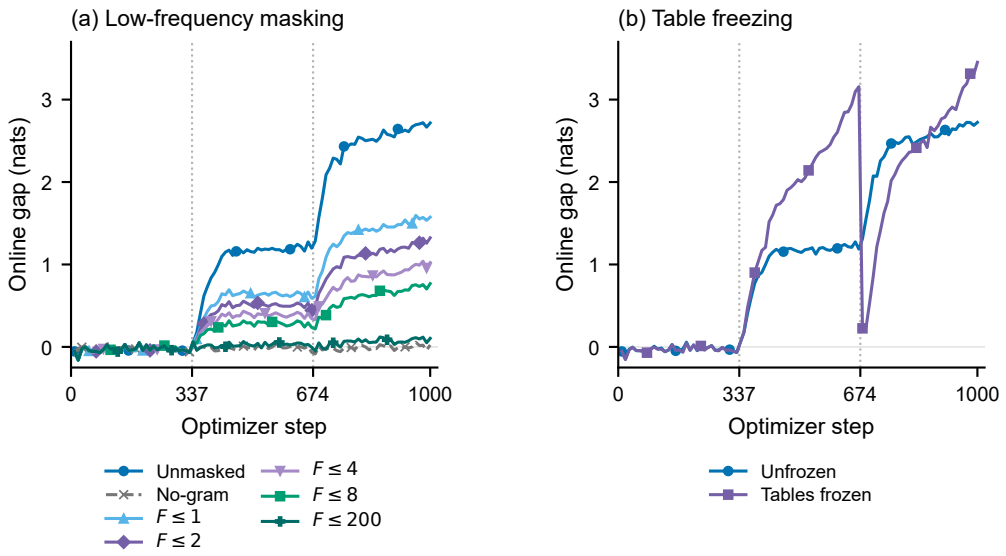


Figure 7: (Left) Online gap under low-frequency retrieval masking. Vertical dotted lines mark replay boundaries; line segments connect raw logged values.

Figure 8: (Right) Online gap with trainable or frozen tables. Vertical dotted lines mark replay boundaries; line segments connect raw logged values.

4 A Minimal Mechanistic Account of Forking

4.1 Gap Decomposition and Frequency Scaling

Since replay reinforces seen continuations but provides no direct evidence for unseen ones, we attribute the validation–training gap mainly to novel-continuation events and test this empirically. Let f_i denote the number of continuations unseen after context c_i in the fixed training corpus and $g_{i,j}$ their effective gap contribution factor at epoch j. One can obtain the leading approximation to the gap:

$$\mathrm{gap}(i,j) \approx \frac{f_i}{F_i}\, g_{i,j}. \tag{3}$$

Motivated by the shared absence of training examples for novel continuations within their respective contexts, we approximate $g_{i,j}$ at epoch j by a common g_j, treating context-specific differences as secondary. To estimate the probability mass of unobserved continuation types after context c_i, we adopt the Good–Turing (G–T) missing-mass estimation (Good, 1953; Gale & Sampson, 1995), which is given by $\hat{f}_i = N_{1,i}$, where $N_{1,i} = \#\{y : n_i(y) = 1\}$ counts continuation types observed exactly once, y denotes a continuation token, and $n_i(y)$ its count after c_i. All counts refer to a single epoch through the fixed training corpus.

Averaging these estimates over contexts with the same training count F yields

$$\hat{f}(F) = \frac{1}{n(F)} \sum_{i:F_i=F} N_{1,i} = \frac{N_1(F)}{n(F)}, \tag{4}$$

where $n(F) = \#\{i : F_i = F\}$ and $N_1(F) = \sum_{i:F_i=F} N_{1,i}$ is their total singleton count. Experimental measurements on natural language data indicate that $\hat{f}(F)$ follows the empirical scaling $\hat{f}(F) = CF^{1-\alpha}$, with $C = 0.981$, $\alpha \approx 0.340$ for bigram contexts and $C = 0.991$, $\alpha \approx 0.383$ for trigram contexts, as shown in Figure 9.

The probe gap for contexts with the same training count F is approximated by the equally weighted mean of their individual gaps, neglecting differences in context coverage and token weights between

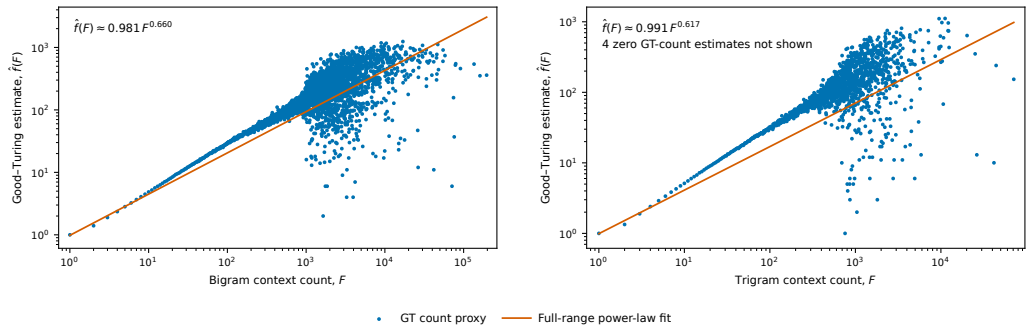


Figure 9: Frequency scaling of the G–T estimate $\hat{f}(F)$ for bigram (left) and trigram (right) contexts. Points show estimates averaged over contexts with the same training count F; solid lines show full-range power-law fits. Four zero estimates in the trigram panel are included in fitting but omitted from the logarithmic display.

the training and validation samples:

$$\mathrm{gap}_j(F) \approx \frac{1}{n(F)} \sum_{i:F_i=F} \mathrm{gap}(i,j) \approx CF^{-\alpha} g_j. \tag{5}$$

As shown in Figure 10, G–T estimate in Equation 5 approximately captures the frequency dependence obtained by directly fitting the measured gaps. For bigram contexts, Equation 5 gives $\mathrm{gap}_{\mathrm{GT}}(F) \approx 9.090F^{-0.340}$ with $g_j = 9.263$, compared with the direct fit $\mathrm{gap}_{\mathrm{fit}}(F) \approx 7.206F^{-0.233}$. For trigram contexts, the corresponding estimate is $\mathrm{gap}_{\mathrm{GT}}(F) \approx 3.996F^{-0.383}$ with $g_j = 4.032$, compared with $\mathrm{gap}_{\mathrm{fit}}(F) \approx 3.759F^{-0.318}$.

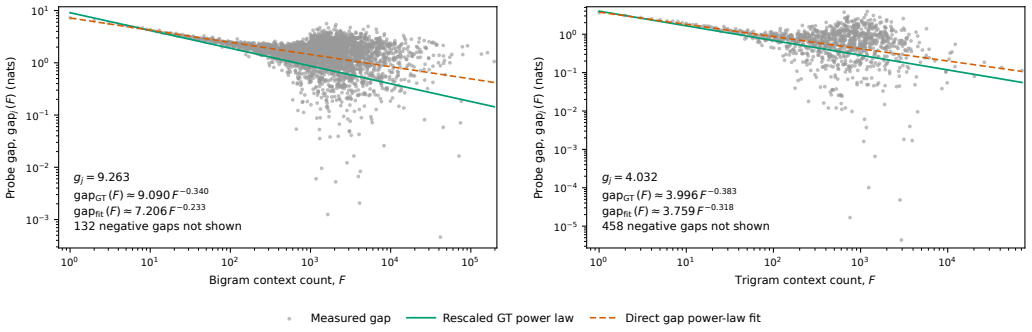


Figure 10: Frequency-resolved probe gaps for bigram (left) and trigram (right) contexts at step 1000, seed 42. Solid lines show $g_j CF^{-\alpha}$ with C and α fixed by the G–T fit; dashed lines fit both amplitude and exponent directly to the gaps. Both fits use the full frequency range with validation token weights on the original gap scale. Negative gaps are included in fitting but omitted from the display.

In addition, G–T estimation does not itself imply the power law in Equation (5). A possible source of this scaling is regular variation in the ranked conditional continuation probabilities, for example $p_{(r)} \propto r^{-s}$, where r ranks continuations in decreasing order of conditional probability for a fixed context. This implies $\alpha = 1 - 1/s$ (Gnedin et al., 2007). Appendix E states the assumptions and the independent measurements needed to test this explanation.

4.2 Logit sharpening under replay

The n-gram table serves two purposes: (i) it partitions the prediction state space into context-specific subspaces and (ii) it promotes logit sharpening during replay. Here, subspace $\mathcal{C}(c_i)$ consists of all

input sequences ending in the n-gram context c_i, while sharpening concentrates predicted probability on observed continuations. To examine local sharpening, we consider a fixed input in $\mathcal{C}(c_i)$ and analyze the cross-entropy gradient.

Consider context c_i for which continuation $v = C$ is observed in training, while $v = D$ is novel. Let a_v denote the logit of v, with probability $p_v = \mathrm{softmax}\, a_v$. For a training target C, the cross-entropy loss is $\ell_C = -\log p_C$. An idealized gradient update at epoch t gives

$$a_v^{(t+1)} - a_v^{(t)} = -\eta \frac{\partial \ell_C}{\partial a_v} = -\eta \left(p_v^{(t)} - \mathbf{1}[v = C] \right), \quad (\eta > 0). \tag{6}$$

Let $z(t) = p_C^{(t)} / p_D^{(t)}$, Equation (6) implies that

$$z(t+1) = z(t) \exp \left[\eta \left(1 - p_C^{(t)} + p_D^{(t)} \right) \right] > z(t). \tag{7}$$

This increasing probability ratio reflects sharpening in favor of the observed continuation C relative to the novel continuation D.

5 Forking in Post-Training

Production pretraining rarely replays data: corpora are large enough that each example is seen once, and Section 3.3 accordingly shows the gap shrinking as the training corpus grows. Post-training inverts both conditions. Supervised fine-tuning often replays small, expensive corpora for several epochs, and reinforcement-learning methods such as GRPO and PPO revisit the same prompts across rollout groups and optimization epochs. This section tests whether forking reappears under exactly these conditions.

Continued training on small replayed corpora. We start from the low-gap 20×-corpus trigram checkpoint (gap 0.214 at step 20,220), reset both optimizers with a fresh 100-step warmup, and replay three 333-batch corpora for five epochs: a same-distribution continuation of the pretraining corpus (ID), Magicoder (code), and OpenR1-Math (math). Each corpus is run with the standard trainable trigram table and with the table frozen while the backbone remains trainable, which is the common production choice. With a trainable table the gap reopens on every source: endpoint gaps are 2.463 (ID), 1.012 (Magicoder), and 0.726 (OpenR1) at seed 42 (Figure 11). Freezing the table substantially reduces but does not remove the effect: validation loss still ends 11% (ID) and 16% (Magicoder) above training loss, and 8% on OpenR1. The full protocol and endpoint table are in Appendix H.

Verifier-selected data as an RL proxy. Verifier-filtered policy optimization concentrates training on accepted answers and repeats each prompt many times. As a static proxy for this regime (RFT or REINFORCE++-style data reuse, not on-policy RL), we replay RLVR-MATH from the Tulu 3 release (Lambert et al., 2024) with the reference answer appended once; each prompt recurs about 8.7 times per epoch, comparable to the per-prompt repetition of group-based sampling. The effect is stronger here than on the SFT corpora: endpoint gaps are 0.612 (trainable) and 0.502 (frozen), so even the frozen-table residual exceeds every SFT source (Figure 12).

Taken together, any post-training stage that replays a narrow corpus can reopen forking in a checkpoint that pretraining left nearly gap-free, and freezing the local memory mitigates but does not prevent it. Forking is therefore a failure mode that replay-heavy post-training pipelines must monitor, not an artifact of our minimal pretraining setting.

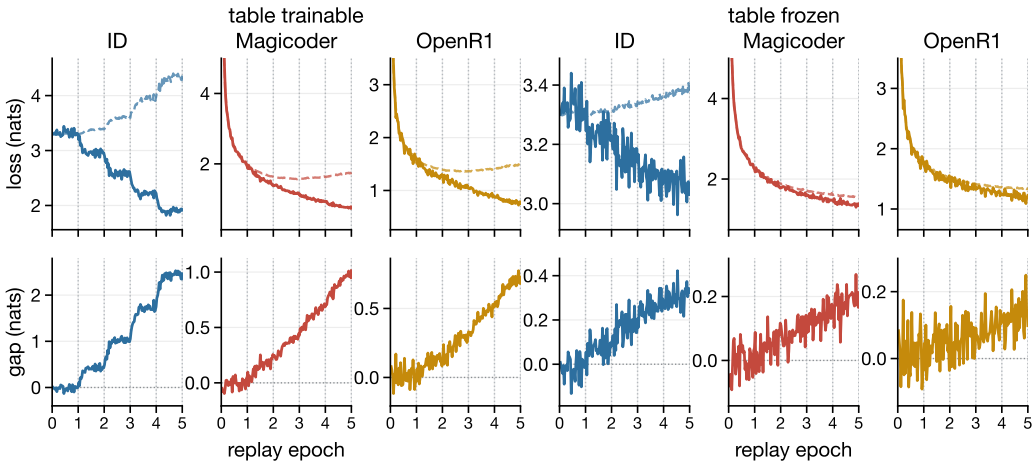


Figure 11: Continued-pretraining stress test: trainable trigram table (left), frozen table (right).

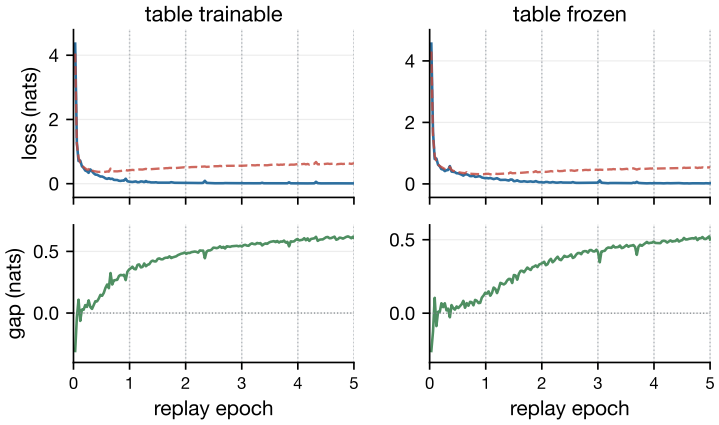


Figure 12: Static RLVR-like proxy on RLVR-MATH.

6 Conclusion and Discussion

An automated architecture search surfaced forking: a replay-aligned divergence between training and validation loss in models with over-encoding n-gram memory. Frequency-resolved measurements tie the gap to incomplete continuation coverage of low-frequency contexts, Good–Turing missing mass predicts its frequency dependence, and replay-driven logit sharpening explains how repeated reinforcement converts coverage gaps into validation damage. The effect reproduces in a DeepSeek-style Engram model (Appendix G) and reopens during continued training and verifier-selected post-training on small replayed corpora (Section 5).

Two practical consequences follow. First, local encoding capacity should be evaluated jointly with continuation coverage and data reuse: a smaller training–validation gap alone is not evidence of better generalization, and frequency-resolved probes expose when local gains come at the expense of uncovered continuations. Second, post-training stages that replay narrow corpora inherit the risk even from a nearly gap-free checkpoint; freezing the local memory mitigates but does not remove it, so replay schedules and gap trajectories deserve monitoring wherever expensive data are reused.

Limitations bound these claims: the transfer results are single-seed, five-epoch measurements at small scale, the RL comparison is a static verifier-selected proxy rather than on-policy policy optimization, and the DeepSeek-style replication tests components jointly rather than attributing the effect to any one of them. Whether response-only instruction tuning or production-scale mixtures damage specific alternative continuations remains open. Methodologically, the workflow used here—anomaly, aligned internal observables, targeted intervention, minimal reconstruction—turns a search-selected modification into a testable scientific question, and underlines that a measured benefit need not transfer across data regimes, architectures, or training stages.

References

Xin Cheng, Rui Tian, Wangding Zeng, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026. URL https://arxiv.org/abs/2601.07372.

William A. Gale and Geoffrey Sampson. Good–turing frequency estimation without tears. *Journal of Quantitative Linguistics*, 2(3):217–237, 1995. doi: 10.1080/09296179508590051. URL https://www.grsampson.net/AGTf.pdf.

Martin Gerlach and Eduardo G. Altmann. Stochastic model for the vocabulary growth in natural languages. *Physical Review X*, 3(2):021006, 2013. doi: 10.1103/PhysRevX.3.021006. URL https://doi.org/10.1103/PhysRevX.3.021006.

Alexander Gnedin, Ben Hansen, and Jim Pitman. Notes on the occupancy problem with infinitely many boxes: General asymptotics and power laws. *Probability Surveys*, 4:146–171, 2007. doi: 10.1214/07-PS092. URL https://arxiv.org/abs/math/0701718.

I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953. doi: 10.1093/biomet/40.3-4.237.

Danny Hernandez, Tom Brown, Tom Conerly, Nova DasSarma, Dawn Drain, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Tom Henighan, Tristan Hume, Scott Johnston, Ben Mann, Chris Olah, Catherine Olsson, Dario Amodei, Nicholas Joseph, Jared Kaplan, and Sam McCandlish. Scaling laws and interpretability of learning from repeated data. *arXiv preprint arXiv:2205.10487*, 2022. URL https://arxiv.org/abs/2205.10487.

Hongzhi Huang, Defa Zhu, Banggu Wu, Yutao Zeng, Ya Wang, Qiyang Min, and Xun Zhou. Over-tokenized transformer: Vocabulary is generally worth scaling. *arXiv preprint arXiv:2501.16975*, 2025. URL https://arxiv.org/abs/2501.16975. ICML 2025.

Andrej Karpathy. nanoGPT. GitHub repository, 2022. URL https://github.com/karpathy/nanoGPT. Accessed 2026-09-16.

Andrej Karpathy. Autoresearch. GitHub repository, 2026. URL https://github.com/karpathy/autoresearch. Accessed 2026-09-16.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2024.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL https://arxiv.org/abs/2408.06292.

Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. *Advances in Neural Information Processing Systems*, 36, 2023. URL https://arxiv.org/abs/2305.16264.

Recursive. First steps toward automated AI research. Technical report, 2026. URL https://www.recursive.com/articles/first-steps-toward-automated-ai-research.

Aurko Roy, Rohan Anil, Guangda Lai, Benjamin Lee, Jeffrey Zhao, Shuyuan Zhang, Shibo Wang, Ye Zhang, Shen Wu, Rigel Swavely, Tao (Alex) Yu, Phuong Dao, Christopher Fifty, Zhifeng Chen, and Yonghui Wu. N-Grammer: Augmenting transformers with latent n-grams. *arXiv preprint arXiv:2207.06366*, 2022. URL https://arxiv.org/abs/2207.06366.

Shanbin Yu, Danni Yu, Baiyu Zhu, Jinxin Wang, Peihang Li, Zirui Peng, Haoran Zhao, Leqian Yang, Yuting He, Shaoyang Guo, Zirui Ren, Qingyu Qu, Chencheng Tang, and Ziming Liu. OPHIS: Towards mechanistic auto-research. MetaCircle Blog, July 2026. URL https://meta-circle.com/blog/ophis-a-new-paradigm-for-autoresearch.

A Original Discovery and Observable Screening

A.1 The original task and discovery setting

NanoGPT RSI searches for changes that reduce validation bits per byte (BPB). An agent modifies `train.py` within a fixed compute budget, evaluates the result, and continues to revise the architecture and training strategy. The original automated search used a training budget of approximately five minutes per experiment. The small-data experiment in which forking was discovered instead used 2000 optimizer steps to inspect a longer trajectory. Forking was an unexpected outcome of this process.

The text came from the Hugging Face dataset `karpathy/climbmix-400b-shuffle`, using shards 1–10 and 6542. In the discovery-stage split, shards 1–3 were used for training, and shards 4–10 and 6542 for validation. This was a stage-specific small-data configuration, not the default split for the entire automated search. The loader read and packed text in a fixed order, then restarted after traversing the training shards. Three shards did not provide enough unique data for 2000 steps, so the run necessarily entered later epochs. Training and validation losses initially decreased together. When text began to repeat, training loss continued to fall while validation loss flattened or increased, producing a rapid divergence near the epoch boundary.

The agent-added n-gram module used independent learnable lookup tables at different layers. Each bigram or trigram context was mapped through two hashes. The two retrieved half-width vectors were concatenated, then injected into the attention value branch through an input-dependent sigmoid gate:

$$v_t^{(\ell)} \leftarrow v_t^{(\ell)} + g_{2,t}^{(\ell)} \odot e_{2,t}^{(\ell)} + g_{3,t}^{(\ell)} \odot e_{3,t}^{(\ell)}. \tag{8}$$

Here $e_{n,t}^{(\ell)}$ is the concatenated lookup result; the gate is computed from the corresponding channels of that layer's input. Hash constants and table parameters differ across layers. The retrieved vectors enter the value branch, rather than being added directly to the token embedding.

Table 1 records the frozen configuration used in the observable-localization experiments below. It differs from the later controlled setting in Section 3.1.

Table 1: Configuration for discovery-stage observable screening. Layer indices start at one.

Component	Configuration
Backbone	8 layers; hidden dimension 768; 6 attention heads; head dimension 128.
Input and batch	Sequence length 2048; 72 sequences or 147456 tokens per update; gradient accumulation 1.
Tokenizer	Shared `tokenizer_fixed_tr10`; vocabulary 8192; built from the first ten shards.
N-gram tables	Two tables per order and injection layer; each has 524288 rows of dimension 384. Bigram at layers 2, 4, 6, 8; trigram at layers 2, 6, 8.
Optimizers	Muon for matrix parameters, base learning rate 0.04; grouped AdamW for token/value embeddings, output and scalar parameters; RMSProp for bigram/trigram tables, learning rate 0.6, second-moment decay 0.999, no weight decay.
Schedule	Archived baseline schedule: Muon and ordinary AdamW warmdown begins at 5% and 35% of the step budget, respectively; final multiplier 0.05. N-gram table learning rates remain constant.
Logging	Four runs, each 1000 steps, seed 42. Validation uses four batches every five steps. Optimizer observables are normally logged every five steps and at every step in the 320–380 window, alongside parameter and layer statistics.

A.2 Localizing the n-gram pathway across controlled replay schedules

A.2.1 Four experiments used to screen observables

We varied when training data were first reused and how reuse was organized, while fixing the model, optimizer configuration, tokenizer, and step budget. The four experiments distinguish ordinary replay, delayed replay, random repetition without a full-pass transition, and an imposed replay cycle.

TR1: sequential replay of one shard. The loader repeatedly traversed shard 1 in a fixed order. The second epoch started at step 338, followed by pronounced forking; the third started at step 687, with a further increase in the gap. This run supplied the boundary event to be localized.

TR2: sequential replay of two shards. The loader used shards 1 and 2 before restarting. At approximately step 350, it was still in the first traversal, and the training and validation curves remained close. The second epoch started at step 683, and the divergence was delayed accordingly. This control distinguishes a response to reuse from a change tied only to the absolute training step.

Random sampling with replacement. Documents were sampled uniformly with replacement from shard 1, then tokenized and packed into batches. Repeated documents could occur from the beginning of training. A gap developed gradually, but there was no full-pass replay transition near step 350. This supplied a control for the local boundary event.

50-new / 50-replay. The loader read 50 consecutive new batches from the shard-1 stream, then replayed those cached batches in the same order. Training loss fell and validation loss rose during replay; the curves moved closer when new data resumed. The resulting cycle of approximately 100 steps provided a test of whether an observable followed the replay state. Near the end of a base-stream epoch, a short block was replayed only for its actual cached length, so the local period could be shorter than 100 steps.

TR1, replacement sampling, and 50/50 used validation shards 2–10 and 6542. TR2 used shards 3–10 and 6542. These validation sets were therefore not identical. Screening compared the temporal responses of internal observables; loss curves identified the event within each run. In this appendix, a step counts completed optimizer updates. The centers 350 and 700 refer to screening windows, while actual epoch boundaries come from loader logs.

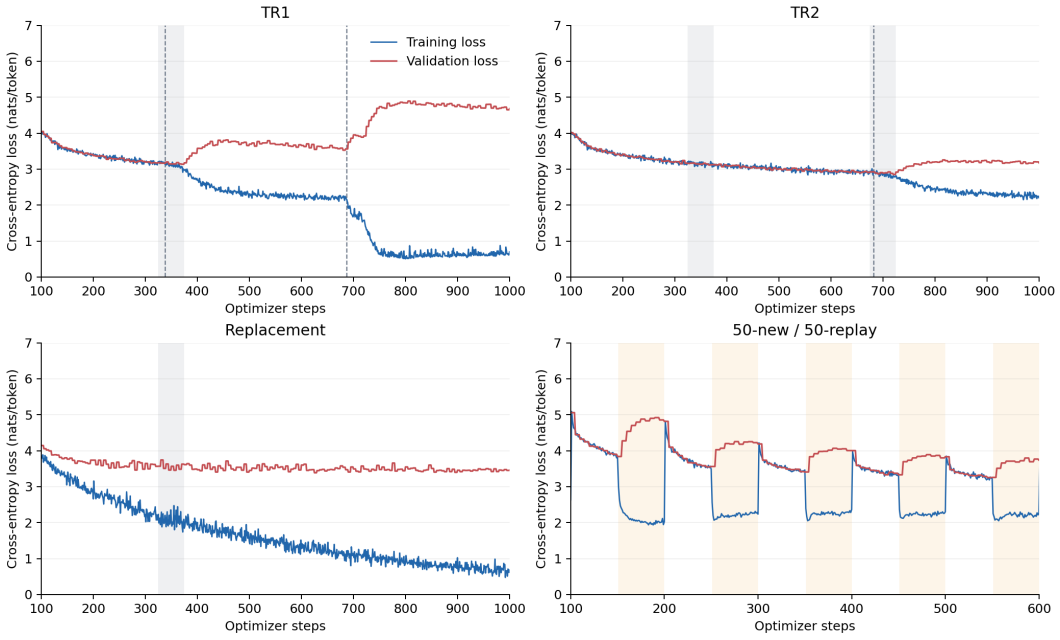


Figure 13: Four data-reuse controls. Blue and red curves show training and validation cross-entropy. Gray shading marks 350 ± 25 and, for TR2, 700 ± 25; dashed lines mark logged epoch boundaries. The 50/50 panel shows steps 100–600, with replay intervals in orange; the other panels show steps 100–1000. Archived curves are unsmoothed. Between validation evaluations, the most recent value is carried forward for display.

A.2.2 FIVE OBSERVABLE-SCREENING RULES

OPHIS records statistics of parameters, gradients, optimizer states, and update vectors, including their directional relationships (Yu et al., 2026). Candidates were scalar time series shared by the four runs. We excluded bookkeeping quantities such as data progress, learning-rate multipliers, throughput, and parameter counts, as well as some auxiliary fields used to construct other statistics. Window calculations used finite recorded values without interpolation.

Let $\mu(X)$ and $\sigma(X)$ denote the mean and population standard deviation of the recorded values in X, let $X \sqcup Y$ concatenate both collections, and set $\varepsilon = 10^{-12}$. To compare windows or runs with different scales, we used

$$D(X,Y) = \frac{|\mu(X) - \mu(Y)|}{\max\{\sigma(X \sqcup Y),\ 0.05|\mu(X \sqcup Y)|,\ \varepsilon\}}. \tag{9}$$

For a local event compared with a neighboring baseline B, we used

$$H(X,B) = \frac{|\mu(X) - \mu(B)|}{\max\{\sigma(X \sqcup B),\ 0.05|\mu(B)|,\ \varepsilon\}}. \tag{10}$$

The windows were $W = [325, 375]$, $L = [255, 315]$, $R = [385, 445]$, and the later TR2 window $W' = [675, 725]$. Write $X_r(W)$ for the values recorded in window W of run r.

C1: a within-run change across replay states. Compare TR2 near steps 350 and 700:

$$C_1 = D\big(X_{\mathrm{TR2}}(W), X_{\mathrm{TR2}}(W')\big) \geq 0.5. \tag{11}$$

The early window is within the first traversal; the later one covers the onset of replay. This rule retains curves that differ across these states.

C2: a cross-run difference at the same training time. Compare TR1 and TR2 near step 350:

$$C_2 = D\big(X_{\mathrm{TR1}}(W), X_{\mathrm{TR2}}(W)\big) \geq 0.5. \tag{12}$$

The runs have the same elapsed optimizer steps and learning-rate schedule, but only TR1 has entered replay.

C3: a boundary event stronger than random repetition. For each run, define

$$E_r = H\big(X_r(W), X_r(L) \sqcup X_r(R)\big). \tag{13}$$

Require

$$C_3 = \max(0, E_{\mathrm{TR1}} - E_{\mathrm{random}}) \geq 0.5. \tag{14}$$

This retains candidates whose local TR1 event exceeds the change at the same time under random repetition.

C4: agreement with the imposed replay period. Within the first 700 steps of the 50/50 run, pair all recorded values separated by exactly 100 steps. Let U, V be the paired columns and ρ_{100} their Pearson correlation. Define

$$a = \frac{\min\{\sigma(U), \sigma(V)\}}{\max\{\sigma(U \sqcup V), \varepsilon\}}, \qquad C_4 = \max(0, \rho_{100}) \min(1, a) \geq 0.15. \tag{15}$$

Only pairs with $t \geq 1$ and $t + 100 \leq 700$ are used. The score is zero when fewer than ten pairs are available.

C5: a center window distinct from both neighbors. Require the TR1 center to differ from both the preceding and following windows:

$$C_5 = \min \left\{ \begin{matrix} H\big(X_{\mathrm{TR1}}(W), X_{\mathrm{TR1}}(L)\big), \\ H\big(X_{\mathrm{TR1}}(W), X_{\mathrm{TR1}}(R)\big) \end{matrix} \right\} \geq 0.5. \tag{16}$$

Taking the smaller deviation rejects curves that differ on only one side.

A candidate had to pass all five thresholds. For manual-review ordering, the script saturated each score as $q_k = C_k/(1 + C_k)$ and ranked candidates by

$$S = 100 \left[0.7 \min_k q_k + 0.3 \ \text{mean}_k \ q_k \right]. \tag{17}$$

The windows and thresholds were exploratory heuristics. These scores narrowed the set for inspection; they were not significance tests or causal evidence.

A.2.3 Manual inspection and component removal

Curves that were identical across the four runs, or differed only by a fixed scale factor, were merged. Representative statistics were chosen for clear parameter-group attribution and interpretability. Manual inspection of the remaining small set highlighted the n-gram table's update magnitude and update–parameter direction relationship. The update magnitude rose after replay began, shifted later in TR2, and followed the alternation between new and replayed data in the 50/50 run (Figure 14). Code inspection confirmed that `ngram_embeddings` referred to lookup parameters in `bigram_ves` and `trigram_ves`. The n-gram pathway therefore became the main candidate for component intervention.

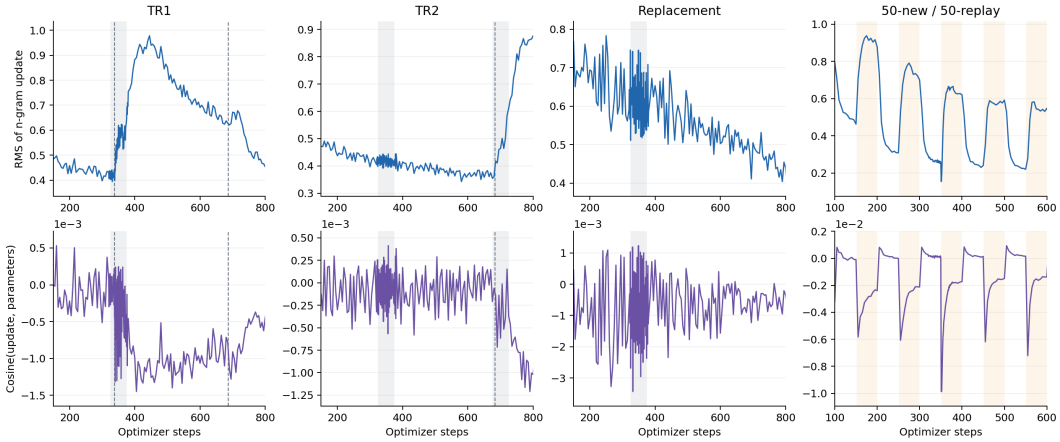


Figure 14: Two representative n-gram observables. The top row shows the RMS magnitude of the RMSProp update vector; the bottom row shows its cosine similarity with the parameter vector. Columns correspond to the four controls. Shading and boundaries follow Figure 13. Each panel uses its own visible vertical range. Unsmoothed curves are used to compare event timing and shape.

We then trained the full model and a model with the bigram/trigram value pathways disabled under the same single-shard sequential-replay setting, 1000-step budget, and seed. Unigram value embeddings were retained. The full model forked sharply after replay began. Disabling the two higher-order pathways removed this pattern, and validation BPB continued to improve in later epochs. Final validation BPB was 1.6769 for the full model and 1.0768 for the intervention. This supports a causal role for the n-gram pathway in this configuration. The disappearing feature was the rapid replay-aligned divergence, not every train–validation difference.

B Standard Setting and Measurement Conventions

B.1 Loss and frequency measurements

The discovery-stage configuration in Appendix A.1 and the controlled setting in Section 3.1 should be distinguished. They use different optimizers and injection implementations. Frequencies $F(i)$ refer to counts in a single unrepeated training corpus, even when that corpus is consumed many times.

For online curves, the training loss is obtained from the training forward pass before the corresponding update, while validation can be evaluated after that update. A fixed-probe gap instead compares fixed

training and validation batches evaluated at the same checkpoint:

$$G_{\text{probe}}(t) = L_{\text{val,probe}}(t) - L_{\text{train,probe}}(t). \tag{18}$$

Where a matched no-gram control is subtracted, we explicitly use

$$G_{\text{net}}(t) = G_{\text{ngram}}(t) - G_{\text{nogram}}(t). \tag{19}$$

Cross-entropy values are in nats unless stated otherwise. BPB is the original search metric and should not be numerically equated with token cross-entropy.

> **Draft note.** Before submission, complete the main-run manifest with exact data and tokenizer fingerprints, table sizes for each capacity condition, batch and probe definitions, checkpoint alignment, and the fitting interval and weighting for each quantitative claim. The source manuscript does not yet provide all of these fields.

B.2 Numerical precision and implementation

The main setting uses BF16 autocast for forward computation. Model weights, n-gram table parameters, and optimizer states remain in FP32; cross-entropy and statistical aggregation are also computed in FP32. Diagnostic forwards use the same BF16 convention as the main evaluation, with per-token losses returned to FP32. Asynchronous aggregation changes execution, not the metric definitions.

FP8 was not used as the baseline. In a setting-validation test, simply selecting `float8_e4m3fn` in CUDA autocast did not work on the current eager `nn.Linear`/`addmm` path. FP8 training requires a dedicated implementation, such as scaled matrix multiplication or Transformer Engine, together with explicit scaling policies for activations, weights, and gradients. Precision boundaries would also have to be specified separately for lookup tables, attention, MLPs, logit projection, table optimization, and diagnostics.

A historical matched-hyperparameter comparison on an H200 found BF16 approximately 5.3 times faster per step than FP32. At 1000 steps, the reported raw gaps were 0.825 for BF16 and 0.858 for FP32, with similar trajectories. This check supported the conclusion that the numerical format did not remove forking in that setting; it is not a substitute for the evidence from the current main runs. The default convention remains BF16 forward computation, FP32 weights and optimizer states, and no `torch.compile`. Any future FP8 comparison should report stability, loss, gap, margins, throughput, and memory use separately.

C Optimization and Implementation Robustness

Table and backbone learning rates should be varied separately to distinguish the speed of establishing a memory representation from later amplification through the backbone. Increasing the learning rate can strengthen an update, but can also enter nonlinear or saturated regimes. It cannot be treated as equivalent to adding replay passes without a model of these responses. The table-freeze and backbone-freeze experiments constrain the roles of the two update paths at different stages.

> **Draft note.** Complete the learning-rate–step relationship, the table-side saturation range, and the backbone response curves. The source notes a saturation tendency around the high table-learning-rate regime, but the quantitative boundary and its theoretical explanation are not yet established here. Separate empirical results from predictions of the eventual model.

D Full Intervention and Replay Curves

Figure 15 and Table 2 retain the main intervention evidence from the Chinese draft. Each intervention addresses a different part of the proposed pathway: context-dependent reads, stable hash assignment, table writing, or backbone adaptation. Interpretations are restricted to the tested thresholds and measurement windows.

The source analysis reports approximately linear aggregate growth over a 20-pass window, with a slope of 0.664 per pass and $R^2 = 0.998$. The 20-pass trajectories appear in the right panels

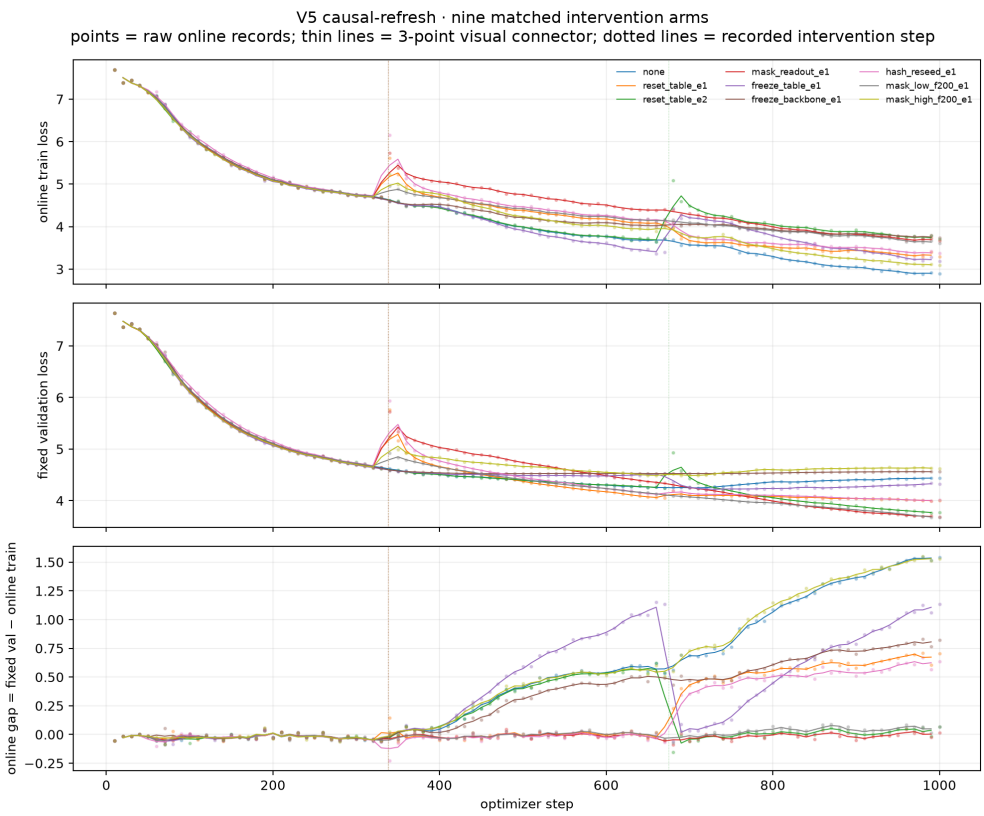


Figure 15: Training, validation, and gap trajectories for the component interventions. The hash-reseeding condition changes the context-to-row mapping at each epoch while retaining table weights and optimizer state. Its reported endpoint raw gap is 0.069.

Table 2: Intervention summary across comparisons in the current draft, not a list of endpoints for Figure 15. Raw gaps and relative reductions have different denominators. Exact run and checkpoint metadata for these source-reported numbers remain to be completed.

Condition	Intervention	Reported observation
No-gram	Remove the n-gram module	Raw gap 0.233; ordinary train–validation differences remain.
N-gram	Input injection; table LR 128×	Raw gap 5.676, with replay-related steps.
Low-frequency mask	Mask low-count reads in training and evaluation	$F \leq 8$ removes about 72% of net gap; higher thresholds further reduce it.
High-frequency mask	Mask high-count reads	Much smaller change than low-frequency masking in the tested comparison.
Hash reseed	Change context-to-row mapping each epoch; retain parameters and optimizer	Raw gap 0.069; substantial suppression of replay accumulation.
Freeze table	Stop table updates from epoch 2	About 94% of second-pass gap remains; growth returns in epoch 3, at a lower level than the two-pass reference.
Freeze backbone	Stop backbone updates from epoch 2	Slightly smaller second-pass gap; little further growth in epoch 3.

Figure pending. The source high-frequency scan uses the old strict condition $F > t$. It is preserved as an asset but must be replaced by the intended $F \geq t$ scan before a threshold-matched comparison is presented.

Figure 16: High-frequency masking: replacement plot pending under the inclusive threshold convention.

of Figure 6; Figure 17 retains a separate, earlier 10-pass comparison. Boundary jumps account for approximately 28% of the accumulated gap in the source analysis, and their share decreases over passes. This motivates a dynamical account that includes within-pass changes, rather than attributing all damage to an instantaneous epoch transition. A finite-window linear fit does not establish unbounded long-run growth.

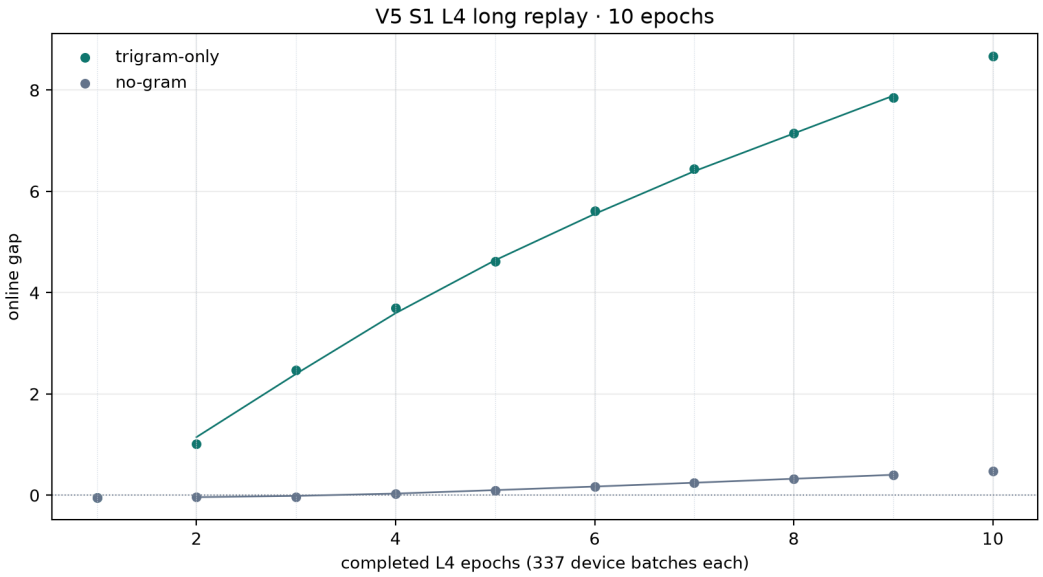


Figure 17: Archived 10-epoch trigram-only replay comparison with a no-gram control. The Chinese source caption describes 20 epochs, but the embedded image covers ten. The image is retained here with its actual scope; it is not the evidence for the 20-pass fit quoted in the text.

A reduced gap must be distinguished from improved validation performance: increasing training loss can shrink the difference without repairing generalization. Likewise, a gap that survives table freezing does not imply that all relevant memory is stored in the table. In the strong-replay comparison recorded in the source, validation loss is approximately 6.95 with n-gram memory and 3.34 without it. Figure 18 compares absolute validation changes relative to the no-gram baseline. In this regime, the additional memory does not provide a net generalization benefit.

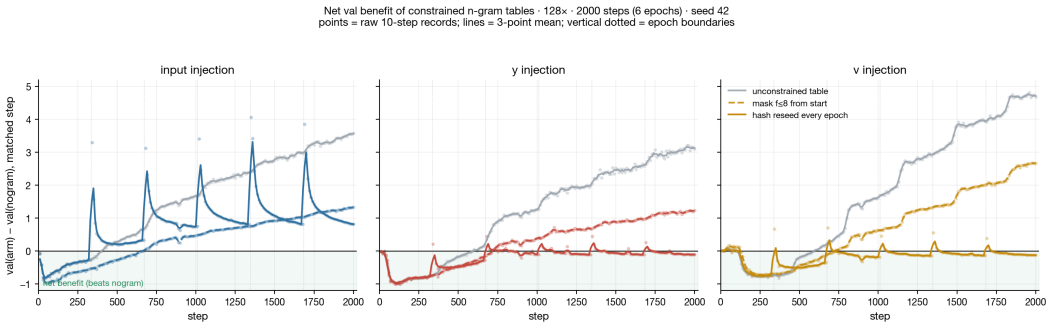


Figure 18: Validation-loss differences relative to the no-gram baseline in the source intervention analysis. These comparisons delimit the benefit of memory in the measured replay regime; they do not establish that n-gram memory should only be used as a short-term cache.

Low-frequency masking and reseeding are mechanism-motivated interventions, but their practical value also depends on preserved training benefits and additional cost. The choice between refreshing memory, retaining it, or protecting selected keys requires evidence under the intended data coverage and training history.

E Mathematical Derivations and Model Tests

E.1 Conditional continuation tails and missing mass

For a fixed context c_i, let $y_{i,(r)}$ denote the r-th most likely continuation and $p_{i,(r)}$ its true conditional probability. For F independent samples $Y_{i,1}, \ldots, Y_{i,F}$ from this distribution, the occurrence count is $n_{i,(r)}(F) = \#\{k \in \{1, \ldots, F\} : Y_{i,k} = y_{i,(r)}\}$. With f_i interpreted as an effective novel-token count, the missing mass satisfies $M_i(F_i) = f_i/F_i$ at the corpus count F_i. Let $K_i(F)$ denote the number of observed continuation types. Then

$$M_i(F) = \sum_r p_{i,(r)} \mathbf{1}\left[n_{i,(r)}(F) = 0\right], \quad K_i(F) = \sum_r \mathbf{1}\left[n_{i,(r)}(F) > 0\right]. \tag{20}$$

Here, $M_i(F)$ is the missing probability mass and $K_i(F)$ is the number of observed continuation types. Their expectations satisfy

$$\begin{aligned} \mathbb{E}[M_i(F)] &= \sum_r p_{i,(r)}(1 - p_{i,(r)})^F, \\ \mathbb{E}[K_i(F+1) - K_i(F)] &= \mathbb{E}[M_i(F)]. \end{aligned} \tag{21}$$

Assume an idealized conditional tail $p_{i,(r)} \sim A_i r^{-s}$ with $s > 1$. Approximating $(1-p)^F$ by e^{-Fp} gives

$$\begin{aligned} \mathbb{E}[M_i(F)] &\sim \int_1^\infty A_i r^{-s} e^{-F A_i r^{-s}} \, \mathrm{d}r \\ &\sim \frac{A_i^{1/s}}{s} \Gamma\left(1 - \frac{1}{s}\right) F^{-(1-1/s)}. \end{aligned} \tag{22}$$

The corresponding coverage growth is

$$\mathbb{E}[K_i(F)] \propto F^{1/s}. \tag{23}$$

Thus, if

$$\mathbb{E}[K_i(F)] \propto F^\theta, \qquad \mathbb{E}[M_i(F)] \propto F^{-\alpha}, \tag{24}$$

then

$$\theta = \frac{1}{s}, \qquad \alpha = 1 - \theta = 1 - \frac{1}{s}. \tag{25}$$

The Good–Turing estimator is

$$\widehat{M}_i(F) = \frac{N_{1,i}(F)}{F}, \tag{26}$$

where $N_{1,i}(F)$ counts continuation types observed exactly once. Its count-scale form is

$$\widehat{f}_i(F) \equiv F \widehat{M}_i(F) = N_{1,i}(F). \tag{27}$$

Since

$$\mathbb{E}[N_{1,i}(F)] = F \, \mathbb{E}[M_i(F-1)], \tag{28}$$

we obtain

$$\mathbb{E}[N_{1,i}(F)] \propto F^{1-\alpha}. \tag{29}$$

This gives the empirical scaling $\widehat{f}(F) \propto F^{1-\alpha}$ used in the main text.

These asymptotics apply to a fixed context over an intermediate frequency range. Finite vocabulary, correlated observations, and context heterogeneity can change the effective exponents. Thus, this is a statistical explanation to be tested from conditional corpus counts and resampling, not a consequence of fitting the gap alone. Corpus-level Zipf/Heaps laws do not by themselves establish the same scaling for conditional continuations (Gnedin et al., 2007; Gerlach & Altmann, 2013).

E.2 Data volume, epoch length, and table capacity

If the context set is fixed, each $F(i)$ is multiplied by k, validation weights remain unchanged, and the per-event damage amplitude is fixed, a missing-mass law with a common exponent gives

$$\sum_i w_i A_j c_i [kF(i)]^{-\alpha} = k^{-\alpha} \sum_i w_i A_j c_i F(i)^{-\alpha}. \tag{30}$$

These assumptions are restrictive. Adding independent data can change the context set, continuation coverage, and the effective frequency distribution. At a fixed step budget, it also changes the number of replays. Fixed-step and fixed-pass comparisons are therefore needed to separate coverage from repeated reinforcement.

Hash collisions may reduce the independence of local memory signals, but they can also mix unrelated contexts. The source does not yet establish how these effects determine the table-capacity exponent.

> **Draft note.** Complete token-mass-weighted aggregation when the frequency distribution changes, the collision model, and matched data-volume experiments. The epoch-length figure is awaiting replacement.

E.3 Further tests of multi-epoch dynamics

> **Draft note.** Derive the optimizer-dependent margin evolution and align it with measured per-pass damage. Distinguish approximately linear accumulation over a finite window, decreasing increments, and a possible long-run plateau. Specify observations that would falsify the model. The local margin, transfer coefficient, and aggregate gap slope currently serve as constraints, not a closed derivation.

F Experiment and Figure Provenance

The English draft is based on revision 2633 of the current Chinese manuscript, retrieved on 16 September 2026. The local source snapshot retains the original section text and figure identifiers. Figure assets were downloaded without changing their data. Legacy figure labels that use f for the context count are explained in captions; the manuscript uses F for that count and f for novel-continuation mass.

The new DeepSeek-like comparison is generated directly from archived fixed-probe logs. The associated script and provenance file record run names, endpoint measurements, and file hashes. The earlier full-model run that stopped at step 1913 is not extended to 2000 steps; the figure instead uses a separately completed diagnostic repeat. No new training was performed for this migration.

> **Draft note.** Complete the run ID, code revision, data split, seed, checkpoint/pass convention, and plotting source for every inherited main and appendix figure. The current local asset manifest identifies the source document blocks, but is not a substitute for a full experimental reproduction manifest.

G Cross-Architecture Replication and Intervention Protocol

Section 3 summarizes the cross-architecture replication; this appendix gives its protocol. The DeepSeek-like configuration is a smaller 12-layer model whose backbone combines MLA-lite attention, a dense SwiGLU first layer, 11 subsequent mixture-of-experts layers with 16 routed experts, top-2 routing and two shared experts, auxiliary-loss-free load balancing, and four-stream mHC. Engram modules at layers 2 and 6 use bigram and trigram retrieval with multi-head hashing, context-dependent gates, and causal local convolution, i.e. multi-layer input over-encoding in the formulation of this paper. It is trained for 2000 updates with seed 42, replaying the fixed shard-1 corpus with a fixed validation probe; the backbone uses AdamW with learning rate 0.0006 and the tables RMSProp with learning rate 0.0768, with 100 warmup steps followed by constant learning rates. The fixed training and validation probes are evaluated at the same checkpoint. The table below lists the archived runs used in the figure; names are included for internal traceability and can be moved to the released artifact manifest for submission.

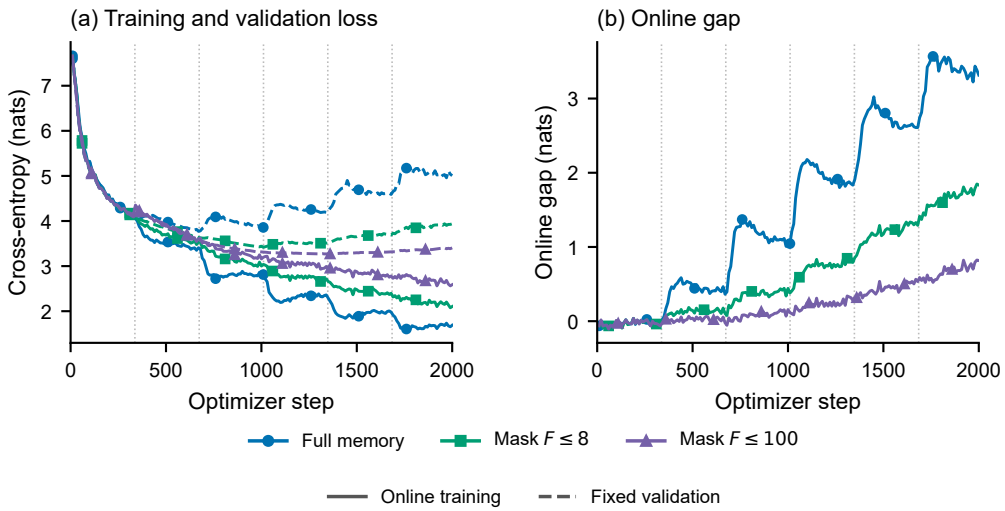


Figure 19: Forking and low-frequency retrieval masking in the DeepSeek-like model. (Left) Online training loss (solid) and fixed validation loss (dashed). (Right) Their difference. Colors and markers identify the unmasked model and two masking thresholds. Vertical dotted lines mark replay boundaries; line segments connect raw logged values.

Table 3: DeepSeek-like runs used for the fixed-probe comparison. All use seed 42.

Condition	Archived run ID
Full repeat	`dscont_v1_full_fault_s42_2k_fixed`
Mask $F \leq 8$	`dscont_v1_maskle8_e2_s42_2k_fixed`
Mask $F \leq 100$	`dscont_v1_maskle100_e2_fault_s42_2k_fixed`
All-read mask	`dscont_v1_maskall_e0_fault_s42_2k_fixed`

The low-frequency conditions suppress retrieved vectors from step 338 onward, according to counts in the original training frequency index. They retain table parameters, optimizer state, and the remaining fusion operations. The completed full-memory and $F \leq 100$ runs include additional non-finite diagnostics. These runs have separate training trajectories and were not resumed from one shared pre-intervention checkpoint. Their agreement is therefore a same-seed, configuration-level comparison, not a bitwise-matched checkpoint intervention.

For the all-read mask, the first failure report records an Inf at step 96 on the input boundary of `blocks.8.ffn.shared_experts.0.down.0`. The run completed 95 finite updates. It is excluded from endpoint performance comparisons and is not treated as a successful no-memory baseline.

A further mechanism stress test could select contexts with several valid continuations, repeatedly train one, and measure the losses of the others. This asks whether local prediction becomes excessively sharp, rather than merely testing a broad distribution shift. Its feasibility and scope require matched experiments. Neither the current replication nor this proposal establishes a vulnerability in a deployed DeepSeek model.

> **Draft note.** Finalize the canonical architecture and data fingerprints, document differences from the minimal setting, and add a stable matched no-memory control. Separate any component-specific claim from the joint-architecture replication.

H CONTINUED-PRETRAINING STRESS TEST

The six runs in Section 5 start from the checkpoint-materialization rerun of the full-pool $20\times$ trigram experiment, with endpoint gap 0.214 at step 20,220. The original scientific run has the same setting and endpoint gap 0.220 but does not retain final weights. We restart the backbone AdamW and table RMSProp states and apply a fresh 100-step warmup.

The model is the vanilla 8-layer, 6-head, 768-dimensional NanoGPT with input trigram injection and a clean 2^{20}-row table. We use seed 42, BF16 forward computation, no compilation, backbone learning rate 6×10^{-4}, table learning rate 0.0768, and fixed validation. Each continued-pretraining source has 333 batches and is run for five epochs (1665 updates); logs are recorded every 10 steps and at the final boundary. The online gap is $G = L_{\mathrm{va}} - L_{\mathrm{tr}}$, where training loss is measured on the current batch before the update and validation loss is measured after the update on fixed batches.

The endpoint measurements are:

Table 4: Endpoint measurements for continued language-model pretraining from the low-gap checkpoint. All values are nats, seed 42, step 1665. The table-trainable and table-frozen runs differ only in whether the trigram lookup table receives gradients.

Source	Table	Train	Validation	Gap
ID	Trainable	1.913	4.376	2.463
ID	Frozen	3.030	3.372	0.341
Magicoder	Trainable	0.740	1.752	1.012
Magicoder	Frozen	1.342	1.555	0.212
OpenR1	Trainable	0.754	1.480	0.726
OpenR1	Frozen	1.226	1.324	0.098

The corresponding dense logs and figures are archived with run IDs `sft_*_e5_freq10_{trainable,frozen}_s42`. These are single-seed continued-pretraining measurements, not response-only instruction-SFT results.

Static RLVR-like proxy. We additionally ran two continued-pretraining runs on `allenai/RLVR-MATH` from the Tulu 3 release (Lambert et al., 2024) (revision `bd2a93551b503a395fadd1a740d957559cfe6f3c`), which contains the 7,500 MATH training problems formatted for RLVR. The prompts carry fixed 4-shot exemplar prefixes that are byte-identical across examples, and the

`ground_truth` field is the reference final answer. The original parquet SHA256 is `2fb8483bf7a4ae9821bee1e715490fb8de5b59f0d2b58bd4de78cc75094d9422`. Messages were retained verbatim and the `ground_truth` was appended once after an "Answer:" marker. The first 95% and last 5% of source examples (in original row order) were used as disjoint train and validation pools, then cycled to 333 train batches and 4 validation batches.

The shared setting matches the six runs above: seed 42, 1665 updates, five replay epochs, fresh optimizer state and 100-step warmup, input trigram-only clean 2^{20}-row table, and fixed validation. In the frozen arm only the table is fixed; the backbone remains trainable. Endpoint measurements are:

Table 5: Static RLVR-like verifier-selected continued-pretraining endpoints. All values are nats at step 1665, seed 42.

Source	Table	Train	Validation	Gap
RLVR-MATH	Trainable	0.009	0.621	0.612
RLVR-MATH	Frozen	0.020	0.522	0.502

The run IDs and dense logs are archived under `rlvr_static_proxy_20260921`. This is a static verifier-selected proxy, not on-policy reinforcement learning.